Software Requirements Specification (SRS)

NS Learnytics - Cloud-Based Academic Analytics and Class Management Platform

Document Version: 1.0.0
Status: Approved / Baselined
Date: September 2026
Standards Compliance: IEEE Std 830-1998 (Recommended Practice for Software Requirements Specifications)
Security Standard: OWASP Top 10 Compliance
Author: Antigravity Engineering Team & System Architects
Project Repository: `NS learnytics`

Table of Contents

1. Introduction (#1-introduction)
    - 1.1 Purpose
    - 1.2 Document Conventions
    - 1.3 Intended Audience & Reading Suggestions
    - 1.4 Project Scope
    - 1.5 References
2. Overall Description (#2-overall-description)
    - 2.1 Product Perspective & Context Architecture
    - 2.2 Product Functions Overview
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design & Implementation Constraints (MySQL 5.1 Compatibility)
    - 2.6 Assumptions and Dependencies
3. Specific System Features & Functional Requirements (#3-specific-system-features--functional-requirements)
    - 3.1 Authentication, Access Control & Profile Management (FR-AUTH)
    - 3.2 Super Administrator Governance Module (FR-ADM)
    - 3.3 Teacher Classroom & Academic Operations Module (FR-TCH)
    - 3.4 Student Academic Portal Module (FR-STU)
    - 3.5 Parent Monitoring & Family Verification Module (FR-PAR)
    - 3.6 Clerk / Operational Moderator Module (FR-CLK)
    - 3.7 Course Scheduling & Enrollment Engine (FR-CLS)
    - 3.8 Dual-Mode Attendance Tracking System (FR-ATT)
    - 3.9 Academic Performance & Marks Recording System (FR-PRF)

    - 3.10 Tuition Fee & Expenditure Financial Engine (FR-FIN)
    - 3.11 Learning Management System & Homework Submission Module (FR-LMS)
    - 3.12 Targeted Announcements & Bidirectional Messaging Engine (FR-MSG)
    - 3.13 AI Risk Radar (Dropout Prediction) Machine Learning Module (FR-AIR)
    - 3.14 AI Grade Forecaster (Linear Regression) Module (FR-AIG)
    - 3.15 Student of the Month Academic Motivation Engine (FR-SOM)
4. External Interface Requirements (#4-external-interface-requirements)
    - 4.1 User Interfaces & Visual Design Standards
    - 4.2 Hardware Interfaces & Scanning Devices
    - 4.3 Software Interfaces & Library Integration
    - 4.4 Communications Interfaces & Protocols
5. Non-Functional Requirements (#5-non-functional-requirements)
    - 5.1 Performance Requirements & Query Optimization
    - 5.2 Safety, Integrity & Duplicate Prevention Requirements
    - 5.3 Security & Role-Based Access Control (RBAC) Requirements
    - 5.4 Software Quality Attributes (Reliability, Usability, Maintainability, Portability)
6. Data Requirements & Database Architecture (#6-data-requirements--database-architecture)
    - 6.1 Logical Data Model & Entity-Relationship Architecture
    - 6.2 Complete Data Dictionary (15 Relational Entities)
7. Machine Learning & Algorithmic Specifications (#7-machine-learning--algorithmic-specifications)
    - 7.1 Random Forest Dropout Predictor Architecture
    - 7.2 Ordinary Least Squares (OLS) Grade Forecaster Architecture
    - 7.3 Student of the Month Ranking Algorithm
8. Verification, Validation & Acceptance Criteria (#8-verification-validation--acceptance-criteria)

---

1. Introduction

1.1 Purpose

The purpose of this document is to provide a complete, formal, and definitive specification of the requirements for the **NS Learnytics** educational management and predictive analytics platform. This Software Requirements Specification (SRS) addresses all functional, non-functional, interface, data, algorithmic, and architectural requirements implemented in the production release. It serves as the foundational technical contract between stakeholders, systems architects, developers, quality assurance engineers, and administrative users.

1.2 Document Conventions

- **Standard:** This specification adheres to the IEEE 830-1998 format.
- **Requirement Identifiers:** Every functional requirement is tagged with a unique identifier in the format `FR-[MODULE]-[NUMBER]` (e.g., `FR-AUTH-01`, `FR-FIN-03`). Non-functional requirements are categorized

as `NFR-[CATEGORY]-[NUMBER]`.

- **Aesthetic Guidelines:** Academic rigor is maintained throughout this document and the software implementation. In compliance with formal institutional standards, no graphical emojis or informal iconography are utilized; standard typographic glyphs, high-contrast labels, and semantic indicators are strictly employed.
- **Priority Terminology:**
  - **SHALL / MUST:** Mandatory requirement; non-negotiable for system deployment.
  - **SHOULD:** Highly recommended requirement; expected in standard operations.
  - **MAY:** Optional feature or future optimization path.

1.3 Intended Audience & Reading Suggestions

This document is structured for the following stakeholders:

- **Software Engineers & DevOps:** Consult Section 2 (Architecture), Section 3 (Functional Specifications), Section 6 (Data Dictionary), and Section 7 (Machine Learning Models).
- **Academic Administrators & Superadmins:** Focus on Section 2.3 (User Classes), Section 3.2 (Super Admin Module), Section 3.10 (Financial Engine), and Section 5 (Security and Reliability).
- **Instructors / Teachers:** Focus on Section 3.3 (Teacher Module), Section 3.8 (Attendance), Section 3.9 (Performance), Section 3.13 (AI Risk Radar), and Section 3.14 (Grade Forecaster).
- **QA & Testing Teams:** Consult Section 3 (Feature Specifications), Section 5 (Non-Functional Requirements), and Section 8 (Acceptance Criteria).

1.4 Project Scope

NS Learnytics is a cloud-ready, responsive, multi-tenant academic operations and predictive learning platform. It replaces fragmented paper-based and siloed digital spreadsheets in private and institutional tutoring centers with an unified portal serving five distinct user roles:

1. **Super Administrator:** Complete institutional control, staff onboarding, user lifecycle management, financial audits, and family relationship validation.
2. **Teacher / Instructor:** Course creation, QR-based and manual attendance scanning, mark books, expenditure tracking, classroom material dissemination, assignment grading, and AI-driven early intervention analytics.
3. **Student:** Academic portal, join-code course enrollment, dynamic QR identification card, homework submission hub, performance trajectory visualizer, fee payment ledger, and peer benchmark tracking.
4. **Parent / Guardian:** Verified student linkage, child attendance tracking, subject-wise score analytics, printable official progress report cards, tuition fee verification, and direct teacher messaging.
5. **Clerk / Moderator:** Daily front-desk physical attendance verification via mobile camera / QR scanner, cash tuition collection, and duplicate billing prevention.

The system incorporates dual machine learning components:

- **AI Risk Radar:** A Random Forest Classifier identifying students at risk of academic failure or course dropout using a 5-dimensional feature vector.

- **AI Grade Forecaster:** An Ordinary Least Squares (OLS) Linear Regression model projecting anticipated examination grades and score trajectories from historical exam and homework milestones.

1.5 References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
2. OWASP Top 10: *The Ten Most Critical Web Application Security Risks*.
3. MySQL 5.1 Reference Manual: *SQL Syntax, Constraints, Subqueries, and Storage Engines (MyISAM/InnoDB)*.
4. Scikit-Learn Documentation: *Random Forest Classifiers and Linear Regression Estimators*.
5. W3C Web Content Accessibility Guidelines (WCAG) 2.1 Level AA.

---

2. Overall Description

2.1 Product Perspective & Context Architecture

NS Learnytics operates as an integrated multi-tier web application built upon a modular Flask architecture communicating with a MySQL relational database management system.

```
+-------------------------------------------------------------------------------+ | CLIENT VIEWPORT | | Responsive Browsers (Desktop PC, Tablet, Mobile Phone) - Vanilla HTML5 / CSS3 | | Chart.js Visualizations | HTML5 QR Code Scanner | AJAX Asynchronous Validation | +-----------------------------------------------------------------------+ | HTTPS / REST JSON | +-------------------------------------------------------------------------------+ | FLASK APPLICATION LAYER | | WSGI Gateway (Gunicorn / Development Server) | | Session Management & Secure Cookie Middleware | | Role-Based Access Control Decorators (@login_required, role validations) | | | | +--------------------+ +---------------------+ +---------------------------+ | | | Core Controllers | | Academic Engines | | Operational Engines | | | | - auth_bp | | - classes_bp | | - attendance_bp | | | | - admin_bp | | - performance_bp | | - finance_bp | | | | - profile_bp | | - lms_bp | | - announcements_bp | | | | - parent_bp | | - enrollment_bp | | - messages_bp | | | +--------------------+ +---------------------+ +---------------------------+ | | | | | +-------------------------------------------------------------------+ | | | INTELLIGENT ANALYTICS SERVICES | | | | - StudentRiskPredictor (Random Forest Classifier: 5-Feature Vector) | | | | - GradeForecaster (Linear Regression OLS Model) | | | | - StudentOfTheMonthService (Aggregation & Tie-Break Cache) | | | +-------------------------------------------------------------------------------+ | +-------------------------------------------------------------------------------+ | PyMySQL Driver | +-------------------------------------------------------------------------------+ | DATABASE STORAGE LAYER | | MySQL 5.1 RDBMS (InnoDB Engine, utf8mb4) | | 15 Relational Tables with Referential Integrity & Compound Unique Constraints | +-------------------------------------------------------------------------------+
```

2.2 Product Functions Overview

- **Identity & Access Management:** User registration, password encryption (PBKDF2/scrypt), session-based role segregation, and personal profile management.
- **Administrative Governance:** Institution-wide analytics, staff provisioning, user archival/deletion with cascade constraints, and parent-student connection auditing.

- **Course & Classroom Orchestration:** Subject cataloging, weekly timetable management, join-code enrollment, and roster auditing.
- **Attendance Verification:** Dual-mode QR code scanning (camera input) and manual fallback search with one-session-per-day enforcement.
- **Mark Book & Academic Records:** Multi-subject assessment recording, percentage conversion, statistical class distribution, and tabular exports (Excel/PDF).
- **Cash & Online Financial Accounting:** Duplicate-guarded tuition fee intake, printable counter receipts, monthly expense tracking, and teacher net income statements.
- **Learning Content & Assignment Management:** File sharing for lecture materials, digital assignment publication, deadline timers, student submissions, and rubric grading.
- **Direct Messaging & Targeted Broadcasts:** Role-targeted announcements (All, Students Only, Parents Only) and two-way unread-tracked messaging between parents and teachers.
- **Predictive Analytics:** Early student dropout classification with diagnostic factor reporting and continuous grade projection curves.
- **Honor Roll:** Automated monthly Student of the Month calculation with dynamic tie-breaking and cross-dashboard publication.

2.3 User Classes and Characteristics

User Class	Technical Privilege Level	Primary Objectives & Operational Context
Super Administrator	Highest (System Superuser)	Creates staff accounts (Teachers, Clerks), creates direct parent accounts, approves or rejects parent connection requests, oversees platform financial volume, and manages database accounts.
Teacher	Elevated (Course Owner)	Defines subjects, sets class fee and schedule, tracks course revenue/expenses, scans attendance, inputs exam marks, publishes assignments, grades submissions, reviews AI Risk alerts, and messages parents.
Student	Standard (Self-Service)	Enrolls in courses using join keys, displays mobile QR identity token, reviews personal attendance records, submits homework online, inspects exam grades, and monitors grade predictions.
Parent	Standard (Auditor / Guardian)	Submits link requests to student IDs, monitors approved student attendance, reviews historical test performance, prints formal academic report cards, and communicates with instructors.
Clerk / Moderator	Operational (Front-Desk)	Scans student QR tokens at entrance gates, registers physical cash tuition payments with duplicate validation, prints financial receipts, and oversees daily check-in workflows.

2.4 Operating Environment

- **Server Operating System:** Linux (Ubuntu 20.04/22.04 LTS, Debian 11/12) or Windows Server 2016+.
- **Runtime Environment:** Python 3.10+ (Standard Library, WSGI compliance).

- **Database Management System:** MySQL 5.1.36+ (fully compatible with legacy 5.1 environments through modern MySQL 8.0+ and MariaDB 10.x).
- **Web Server / WSGI:** Gunicorn 20.x, uWSGI, or Flask internal WSGI for development.
- **Client Browsers:** Modern evergreen browsers supporting HTML5, CSS Grid/Flexbox, ECMAScript 6+, and WebRTC Camera APIs (Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, Microsoft Edge 90+).
- **Viewport Form Factors:** Desktop monitors (1920x1080, 1366x768), Tablet viewports (iPad, Android tablets 768px-1024px), and Smartphone screens (360px-480px width).

2.5 Design & Implementation Constraints (MySQL 5.1 Compatibility)

The software has been engineered to strictly satisfy architectural constraints imposed by legacy deployment environments:

1. **RDBMS Constraints (MySQL 5.1 Compatibility):**
   - Common Table Expressions (`WITH cte AS (...)`) and Window Functions (`ROW_NUMBER() OVER (...)`) are strictly prohibited in SQL queries.
   - All complex multi-level aggregations and rankings (such as Student of the Month rankings and Grade Forecaster timelines) MUST be executed using standard subqueries, ANSI SQL-92 `GROUP BY` clauses, or application-tier Python algorithms.
   - Database engine must default to `InnoDB` to support foreign key referential integrity (`ON DELETE CASCADE`) and transactional isolation (`COMMIT` / `ROLLBACK`).
2. **Visual & Styling Integrity:**
   - Zero emojis across all user interfaces, alerts, badges, modals, and report printouts.
   - Custom CSS Design System (`style.css`) utilizing CSS custom properties for uniform dark slate themes, accessible contrast ratios, and glassmorphic elevation.
3. **Single Codebase Simplicity:**
   - Client scripts utilize Vanilla JavaScript (ES6) with zero heavy frontend framework dependencies (React, Angular, Vue) to guarantee minimal bundle size, instant rendering, and zero compilation overhead.

2.6 Assumptions and Dependencies

- Tutors and students have access to internet-connected devices (smartphones, laptops, or tablets).
- Front-desk clerks utilize a device equipped with a functional web camera or barcode/QR laser scanner for QR verification.
- System time is synchronized across application servers and database servers via Network Time Protocol (NTP) to guarantee exact timestamp consistency.

---

3. Specific System Features & Functional Requirements

3.1 Authentication, Access Control & Profile Management (FR-AUTH)

- **FR-AUTH-01: User Registration:**
   - The system SHALL provide self-service public registration for Students and Parents.
   - The system SHALL reject registration attempts containing duplicate email addresses.

    - The system SHALL assign the role `student` or `parent` based on user selection during registration.
    - Upon student registration, the system SHALL automatically generate a unique 64-character hexadecimal QR token and persist it in `student_profiles`.
- **FR-AUTH-02: Password Cryptography:**
    - The system SHALL encrypt all passwords using secure hashing algorithms (`werkzeug.security.generate_password_hash`) before storage in the `users` table. Plaintext passwords SHALL NOT be stored or logged.
- **FR-AUTH-03: User Authentication & Session Security:**
    - The system SHALL authenticate credentials via email and password hash comparison (`check_password_hash`).
    - Upon successful authentication, the system SHALL establish a server-side session recording `user_id`, `role`, and `user_name`.
    - The system SHALL redirect authenticated users to their corresponding role-specific dashboard (`/admin/dashboard`, `/classes/teacher_dashboard`, `/classes/student_dashboard`, `/parent/dashboard`, `/attendance/clerk_dashboard`).
- **FR-AUTH-04: Role-Based Access Control (RBAC):**
    - The system SHALL enforce route-level authorization guards preventing users from accessing endpoints outside their assigned role, returning an HTTP 403 Forbidden status code upon unauthorized access.
- **FR-AUTH-05: Profile Customization:**
    - The system SHALL allow users to view and update their profile details (Name, Grade, Teacher Subjects, Professional Experience) via `/profile/edit`.

3.2 Super Administrator Governance Module (FR-ADM)

- **FR-ADM-01: Administrative Metrics Dashboard:**
    - The system SHALL calculate and display platform-wide metrics: total enrolled students, total active staff members, total institutional tuition revenue collected, total active classes, and pending parent link requests.
- **FR-ADM-02: Staff Account Provisioning:**
    - The Super Admin SHALL be able to create authorized staff accounts with roles `teacher` or `moderator` (Clerk).
    - When creating a `teacher`, the system SHALL automatically initialize a corresponding `teacher_profiles` record.
- **FR-ADM-03: Direct Parent & Student Provisioning:**
    - The Super Admin SHALL be able to manually create parent accounts and instantly link them with pre-approved status to a specific student ID.
    - The Super Admin SHALL be able to manually create student accounts with specified academic grades and auto-generated QR tokens.
- **FR-ADM-04: Parent-Student Relationship Auditing:**

    - The Super Admin SHALL view all pending parent verification requests showing parent name, parent email, target student name, and student email.
    - The Super Admin SHALL have sole authority to execute `approve` or `reject` actions on pending parent-student links.
- **FR-ADM-05: User Lifecycle & Cascade Removal:**
    - The Super Admin SHALL be able to permanently delete any staff or student user account.
    - The system SHALL prohibit a Super Admin from deleting their own active session account.
    - Upon user deletion, the system SHALL execute referential cleanup across profiles, enrollments, attendance records, marks, payments, and parent links.

3.3 Teacher Classroom & Academic Operations Module (FR-TCH)

- **FR-TCH-01: Teacher Executive Dashboard:**
    - The system SHALL present an integrated dashboard displaying active courses, total enrolled student count, gross fee earnings, and recent institutional announcements.
    - The dashboard SHALL render a dedicated "Today's Schedule" widget listing classes scheduled for the current day of the week.
    - The dashboard SHALL display a "Grade Spread" analytics widget depicting overall grade distribution across all taught courses.
- **FR-TCH-02: Class Timetable & Fee Configuration:**
    - The Teacher SHALL be able to create and configure classes by specifying Subject Name, Weekly Schedule (e.g., "Monday 4:00 PM"), and Monthly Tuition Fee.
    - The system SHALL automatically generate a cryptographically unique uppercase 6-character alphanumeric `join_code` for each new class.
- **FR-TCH-03: Course Roster & Student Management:**
    - The Teacher SHALL inspect class rosters detailing enrolled student names, emails, academic grades, and enrollment timestamps.
    - The Teacher SHALL be empowered to drop/unenroll a student from their class roster.
- **FR-TCH-04: Expenditure Recording & Net Financials:**
    - The Teacher SHALL log operational teaching expenditures (e.g., printing costs, hall rentals, equipment) with description, amount, and date.
    - The system SHALL aggregate gross tuition fees collected against operational expenditures to compute the teacher's net operating income.

3.4 Student Academic Portal Module (FR-STU)

- **FR-STU-01: Student Personal Dashboard:**
    - The student dashboard SHALL display enrolled courses, overall attendance rate (%), total assessments completed, and total tuition fees paid.
    - The dashboard SHALL render a prominent "Student of the Month" honor card displaying the top achiever for each enrolled subject.
- **FR-STU-02: Dynamic Digital QR Identity Card:**

- The system SHALL provide a dedicated `/attendance/my_qr` portal rendering a live QR code representing the student's unique profile token.
- The QR interface SHALL include an instant Print Action for physical badge generation.

- **FR-STU-03: Self-Enrollment via Join Code:**
  - The student SHALL be able to enter a 6-character alphanumeric join code to self-enroll into any active class.
  - The system SHALL reject duplicate enrollments and invalid join codes with informative feedback.
- **FR-STU-04: Academic Trajectory & Forecast Inspection:**
  - The student SHALL view their personal exam history, test dates, percentage scores, and AI-predicted next-exam scores per enrolled course.

3.5 Parent Monitoring & Family Verification Module (FR-PAR)

- **FR-PAR-01: Student Link Initiation:**
  - A registered Parent SHALL be able to request linkage to a student by submitting the student's registered email address.
  - The system SHALL place valid requests into a `pending` state awaiting Super Admin review.
- **FR-PAR-02: Unified Multi-Ward Dashboard:**
  - The Parent dashboard SHALL display summary cards for all approved linked children, allowing the parent to toggle between children.
- **FR-PAR-03: Comprehensive Academic & Attendance Monitoring:**
  - For each approved child, the parent SHALL inspect overall attendance percentage, recent absence events, subject test marks, and teacher remarks.
- **FR-PAR-04: Printable Progress Report Card:**
  - The parent portal SHALL integrate a dedicated "Progress Report" view accessible via the persistent dashboard navigation sidebar.
  - The progress report SHALL present student profile metadata, cumulative attendance summary, subject-wise academic marks breakdown, and AI trajectory notes.
  - The report SHALL include print styling (`@media print`) stripping away navigation chrome and sidebars to produce a clean, formal document suitable for institutional presentation.

3.6 Clerk / Operational Moderator Module (FR-CLK)

- **FR-CLK-01: Front-Desk Check-in Console:**
  - The system SHALL provide a specialized Clerk portal (`/attendance/clerk_dashboard`) optimized for high-throughput gate check-in and tuition intake.
- **FR-CLK-02: Dual-Mode Attendance Capture:**
  - The clerk SHALL be capable of verifying attendance via live camera QR code scanning or by manual entry of student ID / QR token string.
- **FR-CLK-03: Duplicate-Guarded Front-Desk Tuition Collection:**
  - The clerk SHALL collect monthly tuition payments for any student and class.

- The clerk UI SHALL perform client-side and server-side duplicate checks preventing intake if the selected student has already paid for the target billing period.
- The system SHALL generate an instant, printable counter receipt upon successful payment registration.

3.7 Course Scheduling & Enrollment Engine (FR-CLS)

- **FR-CLS-01: Unique Class Join Code:**
  - Every class created in the system SHALL possess a globally unique join code stored in `classes.join_code`.
- **FR-CLS-02: Enrollment Referential Integrity:**
  - The system SHALL maintain enrollment records in the `enrollments` table with a compound unique constraint `UNIQUE(class_id, student_id)`.
  - Attempts to re-enroll an already enrolled student into the same class SHALL be idempotently handled or rejected.

3.8 Dual-Mode Attendance Tracking System (FR-ATT)

- **FR-ATT-01: Attendance Recording Protocols:**
  - The system SHALL record attendance records in the `attendance` table with attributes: `class_id`, `student_id`, `date`, and `status` (`present` or `absent`).
  - The system SHALL enforce a database-level uniqueness constraint `UNIQUE(class_id, student_id, date)` preventing duplicate daily logs.
- **FR-ATT-02: Real-time Camera QR Scanner:**
  - The attendance capture interface SHALL integrate the HTML5 QR Code scanning library to process camera frames in real time, decode student tokens, and transmit an asynchronous check-in request to `/attendance/mark_qr`.
- **FR-ATT-03: Manual Fallback Check-in:**
  - In cases where camera hardware is unavailable or student physical cards are damaged, the operator SHALL enter the student's identifier into a manual search field to mark attendance.
- **FR-ATT-04: Attendance Data Export:**
  - Instructors and administrators SHALL be able to export class attendance sheets in Microsoft Excel (`.xlsx`) format and Adobe PDF format via `/attendance/export`.

3.9 Academic Performance & Marks Recording System (FR-PRF)

- **FR-PRF-01: Assessment Entry:**
  - The Teacher SHALL enter marks for any class assessment by specifying Test Name, Max Marks (default 100), Date Recorded, and Marks Obtained for each enrolled student.
- **FR-PRF-02: Statistical Aggregation:**
  - The system SHALL calculate individual percentage scores (`marks_obtained / max_marks * 100`), class averages, maximum scores, and minimum scores.
- **FR-PRF-03: Performance Data Export:**

- The system SHALL provide automated export endpoints generating structured Excel spreadsheets and formatted PDF transcripts of examination results.

3.10 Tuition Fee & Expenditure Financial Engine (FR-FIN)

- **FR-FIN-01: Relational Payment Ledger:**
  - The system SHALL record fee payments in the `payments` table with attributes: `class_id`, `student_id`, `amount`, `payment_date`, and `period` (formatted as `YYYY-MM`).
- **FR-FIN-02: Strict Duplicate Payment Prevention:**
  - **Database Level:** The `payments` table SHALL enforce a compound unique key `UNIQUE KEY unique_class_student_period (class_id, student_id, period)`.
  - **Server Level:** The `/finance/collect_fee` route SHALL query the `payments` table for an existing record matching `(class_id, student_id, period)` before inserting. If detected, the server SHALL abort the transaction and return an explicit HTTP 409 Conflict error.
  - **Client Level:** The fee collection interface SHALL execute an asynchronous AJAX pre-check (`/finance/check_duplicate_payment`) whenever Class, Student, or Period selections change. If a payment exists, the system SHALL display a warning banner and disable the submission button.
- **FR-FIN-03: Printable Payment Receipts:**
  - The system SHALL provide an instant printable counter receipt via `/finance/receipt/<payment_id>` displaying receipt number, transaction timestamp, student name, subject, billing month, amount paid, and cashier verification details.

3.11 Learning Management System & Homework Submission Module (FR-LMS)

- **FR-LMS-01: Digital Course Materials Sharing:**
  - Teachers SHALL upload instructional files (PDF, DOCX, PPTX, TXT) associated with a specific class.
  - Enrolled students and their linked parents SHALL download instructional files via secure download endpoints.
- **FR-LMS-02: Assignment Publication:**
  - Teachers SHALL publish assignments with a title, instructions, due date/time, and maximum points.
- **FR-LMS-03: Digital Homework Submission:**
  - Enrolled students SHALL submit assignments prior to the deadline, providing submission notes and/or uploading digital attachments.
  - The system SHALL enforce a single submission record per student per assignment via `UNIQUE(assignment_id, student_id)`.
- **FR-LMS-04: Homework Grading & Feedback:**
  - Teachers SHALL inspect submissions, assign numerical marks, write narrative feedback, and update submission status to `graded`.

3.12 Targeted Announcements & Bidirectional Messaging Engine (FR-MSG)

- **FR-MSG-01: Targeted Course Announcements:**
  - Teachers SHALL post announcements targeted to: `all`, `students_only`, or `parents_only`.

- The system SHALL filter announcement visibility based on the active user's session role.

- **FR-MSG-02: Direct Parent-Teacher Messaging:**
  - The system SHALL provide private, two-way messaging between Teachers and Parents linked to common students.
  - The system SHALL track message status (`is_read` , `is_edited` , `is_deleted`) and display real-time unread badges.

3.13 AI Risk Radar (Dropout Prediction) Machine Learning Module (FR-AIR)

- **FR-AIR-01: Algorithmic Architecture:**
  - The system SHALL implement a Scikit-Learn `RandomForestClassifier` trained on normalized academic and attendance indicators.
- **FR-AIR-02: Feature Vector Specification:**
  - The prediction engine SHALL construct a 5-dimensional feature vector:
    1. `attendance_pct` : Lifetime attendance percentage across all enrolled classes.
    2. `recent_att_pct` : Attendance percentage over the preceding 30 calendar days.
    3. `avg_mark_pct` : Weighted mean percentage across all assessment marks.
    4. `mark_trend` : Delta between recent score averages and early score averages.
    5. `missed_hw_ratio` : Ratio of unsubmitted LMS assignments against total published assignments.
- **FR-AIR-03: Risk Tier Classification:**
  - The model SHALL output calibrated class probabilities mapped to a composite risk percentage (0.0% to 100.0%).
  - **HIGH Risk:** Risk score $\geq 55\%$, or lifetime attendance $< 55\%$, or average marks $< 45\%$.
  - **MEDIUM Risk:** Risk score between 25% and 54.9%, or attendance $< 75\%$, or average marks $< 60\%$.
  - **LOW Risk:** Risk score $< 25\%$ with satisfactory attendance and academic metrics.
- **FR-AIR-04: Actionable Diagnostic Reporting:**
  - The engine SHALL generate natural-language diagnostic bullet points detailing the specific root factors triggering the risk tier (e.g., "Critical overall attendance (48.2%)", "Declining score trend (-16.5% trajectory)").

3.14 AI Grade Forecaster (Linear Regression) Module (FR-AIG)

- **FR-AIG-01: Chronological Score Trajectory Mapping:**
  - The forecaster SHALL assemble an indexed historical sequence combining examination scores and graded LMS assignment scores sorted chronologically.
- **FR-AIG-02: Ordinary Least Squares (OLS) Modeling:**
  - For subjects with ≥ 2 historical evaluations, the system SHALL fit an OLS Linear Regression model ($y = mx + c$) over the chronological index sequence to project the expected score for the subsequent examination ($x = n + 1$).
- **FR-AIG-03: Boundary Constraints & Trajectory Classification:**
  - Projected scores SHALL be clipped within the closed interval $[0.0, 100.0]$.

    - The model SHALL evaluate the slope coefficient (m):
        - **Improving Trajectory:** $m > +1.5$
        - **Declining Trajectory:** $m < -1.5$
        - **Stable Performance:** $-1.5 \leq m \leq +1.5$
- **FR-AIG-04: Letter Grade Translation:**
    - The system SHALL translate projected percentages into standardized letter grades:
        - **A:** $\geq 85\%$
        - **B:** $75\% - 84.9\%$
        - **C:** $65\% - 74.9\%$
        - **S (Simple Pass):** $50\% - 64.9\%$
        - **F (Fail):** $< 50\%$

3.15 Student of the Month Academic Motivation Engine (FR-SOM)

- **FR-SOM-01: Monthly Top Achiever Evaluation:**
    - The system SHALL identify the top-performing student for each academic subject based on the highest average test score (`AVG(marks_obtained / max_marks * 100)`) achieved during the evaluation month period (`YYYY-MM`).
- **FR-SOM-02: Tie-Breaking Hierarchy:**
    - In the event of identical percentage scores, the system SHALL break ties in favor of the student with the higher count of completed tests.
- **FR-SOM-03: Dynamic Fallback Resolution:**
    - If no assessment marks are recorded in the active calendar month (e.g., beginning of month or holiday break), the engine SHALL automatically resolve to the most recent historical month containing recorded assessment marks.
- **FR-SOM-04: Persistent Cache & Cross-Dashboard Publication:**
    - Evaluated honorees SHALL be saved to the `student_of_the_month` table (`UNIQUE KEY unique_month_subject (month_period, subject)`).
    - The current honorees SHALL be prominently rendered across the Student Dashboard and Teacher Dashboard with honor badge styling.

---

4. External Interface Requirements

4.1 User Interfaces & Visual Design Standards

- **Design Philosophy:** Premium dark slate SaaS aesthetic utilizing clean geometric typography (system-ui, Inter, sans-serif), CSS custom properties, responsive flex/grid layouts, and glassmorphic translucent panels.
- **Accessibility:** High contrast ratios satisfying WCAG 2.1 AA specifications.
- **Print Optimization:** Dedicated print media queries (`@media print`) for Parent Progress Reports and Payment Counter Receipts, hiding sidebars, header navigation bars, and background colors to deliver

printer-friendly documents.

- **Responsive Breakpoints:**
  - Mobile Phones: Viewport width $\leq 640\text{px}$.
  - Tablets & Small Laptops: Viewport width between 641px and 1024px.
  - Desktop Displays: Viewport width $> 1024\text{px}$.

4.2 Hardware Interfaces & Scanning Devices

- **Optical Webcams / Mobile Cameras:** Utilized by Front-Desk Clerks and Instructors via the `Html5QrcodeScanner` API for instant student identification.
- **Laser / 2D Barcode Scanners:** Supported via standard USB keyboard emulation (HID mode) entering decoded QR token strings directly into manual input fields.
- **Point-of-Sale (POS) & Standard Printers:** Supported for generating standard 80mm receipts and A4 academic progress reports.

4.3 Software Interfaces & Library Integration

- **Relational Database:** MySQL 5.1+ via the `PyMySQL` driver executing parameter-bound SQL statements.
- **Data Export Utilities:**
  - `openpyxl` (version 3.0+): Structured generation of formatted `.xlsx` spreadsheets for attendance rosters and grade sheets.
  - `reportlab` (version 3.6+): Dynamic rendering of vector-accurate `.pdf` documents and report cards.
- **Machine Learning Toolkits:**
  - `scikit-learn` & `numpy`: Model definition, synthetic training data bootstrapping, matrix operations, and regression inference.
- **Frontend Charting Engine:**
  - `Chart.js` (v3.x / v4.x): Dynamic canvas-based rendering of score trends, attendance bars, and grade distribution charts.

4.4 Communications Interfaces & Protocols

- **Transport Security:** HTTP/1.1 and HTTP/2 over TLS 1.3 (HTTPS) protecting session identifiers, credentials, and academic records.
- **Asynchronous Data Exchange:** RESTful JSON endpoints utilizing the standard `fetch()` API for client-server communication (e.g., real-time QR token resolution, duplicate payment validations).

---

5. Non-Functional Requirements

5.1 Performance Requirements & Query Optimization

- **NFR-PERF-01: API Latency:** Core transactional endpoints (attendance marking, mark entry, fee collection) SHALL respond within $\leq 250\text{ms}$ under standard concurrent operating loads (up to 100 concurrent requests).

- **NFR-PERF-02: Analytics Inference Time:** Model inference for the AI Risk Radar and AI Grade Forecaster SHALL complete in $\leq 50\text{ms}$ per student.
- **NFR-PERF-03: MySQL 5.1 Optimization:** All SQL queries SHALL avoid unindexed table scans on large tables (`attendance`, `marks`, `payments`) by anchoring operations on foreign keys and compound unique indexes.

5.2 Safety, Integrity & Duplicate Prevention Requirements

- **NFR-SAFE-01: Idempotent Payment Collection:** The system SHALL guarantee that no student can be billed twice for the same class in the same calendar month under any circumstance, enforced by the compound database key `unique_class_student_period`.
- **NFR-SAFE-02: Attendance Double-Count Guard:** The system SHALL prevent multiple attendance marks for the same student in the same class on the same date via `unique(class_id, student_id, date)`.
- **NFR-SAFE-03: Transaction Atomicity:** Financial recordings and user deletion procedures SHALL execute inside explicit database transactions, rolling back all partial updates upon runtime exceptions.

5.3 Security & Role-Based Access Control (RBAC) Requirements

- **NFR-SEC-01: SQL Injection Immunity:** All database queries without exception SHALL use parameterized query substitution (`%s` placeholders with parameter tuples via PyMySQL). Dynamic string concatenation in SQL statements is strictly prohibited.
- **NFR-SEC-02: Session Hijacking Mitigation:** Session cookies SHALL be configured with `HttpOnly`, `SameSite=Lax`, and `Secure` (in HTTPS production) flags.
- **NFR-SEC-03: Access Control Enforcement:** Direct URL tampering (Insecure Direct Object Reference - IDOR) SHALL be actively mitigated by verifying that requesting teachers own the queried class and requesting parents possess approved link status to the requested student.

5.4 Software Quality Attributes

- **Usability:** The interface SHALL provide immediate visual feedback upon user actions (loading spinners, alert messages, color-coded status badges).
- **Reliability:** The system SHALL achieve an uptime of $\geq 99.5\%$ during institutional operating hours.
- **Maintainability:** Modularity SHALL be preserved through Flask Blueprints, separating concerns into discrete controllers (`auth`, `admin`, `attendance`, `performance`, `finance`, `lms`, `messages`, `parent`, `ai_analytics`).
- **Portability:** The application SHALL run across heterogeneous host environments (Linux VPS, Windows Server, Docker containers, Vercel Serverless WSGI).

---

6. Data Requirements & Database Architecture

6.1 Logical Data Model & Entity-Relationship Architecture

The NS Learnytics schema comprises 15 relational tables engineered to maintain absolute referential integrity using foreign keys with cascading deletes.

```
+------------------+ | users | +------------------+ / | \ / | \ v v v +-----------------+ +------
--------+ +---------------------+ | teacher_profiles| |student_profiles |parent_student_links| +--
---------------+ +---------------+ +---------------------+ | | | v | | +-----------+ | | | classes
| | | +-----------+ | | / | \ | | v v v v v +-------------+ +-------------+ +---------------+ +---
---------------+ | enrollments | | attendance | | marks | | payments | +-------------+ +----------
---+ +---------------+ +------------------+ | | | | +---------------+----------------+-----------
-----+ | v +---------------------+ | student_of_the_ | | month | +---------------------+ Additional
Supporting Entities: - expenses (teacher operational costs) - materials (class lecture notes &
resources) - assignments & assignment_submissions (LMS homework workflow) - announcements (role-
filtered class broadcasts) - messages (two-way teacher-parent communications) - notifications
(user alert notifications)
```

6.2 Complete Data Dictionary (15 Relational Entities)

1. **`users`**

Primary authentication and identity entity.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Unique user identifier
`name`	VARCHAR(100)	No		Full name of the user
`email`	VARCHAR(100)	No	Unique	Unique email address for login
`password_hash`	VARCHAR(255)	No		Cryptographic hash of password
`role`	ENUM('teacher', 'student', 'moderator', 'superadmin', 'parent')	No		Access control role
`created_at`	TIMESTAMP	No	Default Now	Account creation timestamp

2. **`teacher_profiles`**

Extended profile for instructional staff.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`user_id`	INT	No	FK	Reference to `users.id` (ON DELETE CASCADE)
`subjects`	VARCHAR(255)	Yes		Comma-separated list of teaching subjects
`experience`	TEXT	Yes		Professional biography and qualifications

3. **`student_profiles`**

Extended profile for enrolled students.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`user_id`	INT	No	FK	Reference to `users.id` (ON DELETE CASCADE)
`grade`	VARCHAR(50)	Yes		Academic grade level (e.g., "Grade 11")
`qr_code`	VARCHAR(64)	Yes	Unique	Cryptographic hexadecimal token for QR badge

4. **`classes`**

Classroom and course definitions.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`teacher_id`	INT	No	FK	Reference to `users.id` (Instructor)
`subject`	VARCHAR(100)	No		Course / subject title
`schedule`	VARCHAR(100)	No		Weekly timetable schedule description
`fee`	DECIMAL(10,2)	No		Monthly tuition fee amount
`join_code`	VARCHAR(20)	No	Unique	Alphanumeric 6-character course join code
`created_at`	TIMESTAMP	No	Default Now	Class creation timestamp

5. **`enrollments`**

Course enrollment mapping.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK, Unique(1)	Reference to `classes.id`
`student_id`	INT	No	FK, Unique(2)	Reference to `users.id`
`joined_at`	TIMESTAMP	No	Default Now	Enrollment timestamp
Constraint			**UNIQUE(class_id, student_id)**	Prevents duplicate student enrollments

6. **`attendance`**

Daily attendance tracking logs.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK, Unique(1)	Reference to `classes.id`
`student_id`	INT	No	FK, Unique(2)	Reference to `users.id`
`date`	DATE	No	Unique(3)	Date of class session
`status`	ENUM('present', 'absent')	No		Verification status
Constraint			**UNIQUE(class_id, student_id, date)**	Enforces one attendance record per day

7. **`marks`**

Assessment marks and exam records.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK	Reference to `classes.id`
`student_id`	INT	No	FK	Reference to `users.id`
`test_name`	VARCHAR(100)	No		Examination / assessment title
`marks_obtained`	DECIMAL(5,2)	No		Score achieved by the student
`max_marks`	DECIMAL(5,2)	No	Default 100.00	Maximum possible score
`date_recorded`	DATE	No		Examination administration date

8. **`payments`**

Tuition fee collection records with duplicate barrier.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK, Unique(1)	Reference to `classes.id`
`student_id`	INT	No	FK, Unique(2)	Reference to `users.id`
`amount`	DECIMAL(10,2)	No		Fee amount paid
`payment_date`	DATE	No		Date transaction was recorded
`period`	VARCHAR(20)	No	Unique(3)	Billing month period (`YYYY-MM`)
Constraint			**UNIQUE KEY unique_class_student_period (class_id, student_id, period)**	Prevents duplicate billing

9. **`expenses`**

Teacher operational expenditures.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`teacher_id`	INT	No	FK	Reference to `users.id`
`description`	VARCHAR(255)	No		Nature of expenditure
`amount`	DECIMAL(10,2)	No		Expense amount
`expense_date`	DATE	No		Date expenditure occurred

10. **`parent_student_links`**

Family relationship mapping with verification lifecycle.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`parent_id`	INT	No	FK, Unique(1)	Reference to `users.id` (Parent)
`student_id`	INT	No	FK, Unique(2)	Reference to `users.id` (Student)
`status`	ENUM('pending', 'approved', 'rejected')	No	Default 'pending'	Administrative verification state
`created_at`	TIMESTAMP	No	Default Now	Request submission timestamp
Constraint			**UNIQUE(parent_id, student_id)**	Prevents duplicate linkage requests

11. **`notifications`**

System alerts and direct notifications.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`user_id`	INT	No	FK	Target recipient `users.id`
`message`	TEXT	No		Notification body text
`type`	VARCHAR(50)	Yes	Default 'general'	Alert category
`is_read`	BOOLEAN	Yes	Default FALSE	Read status flag
`created_at`	TIMESTAMP	No	Default Now	Alert generation timestamp

12. **`messages`**

Two-way direct communication between parents and instructors.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`sender_id`	INT	No	FK	Sender `users.id`
`receiver_id`	INT	No	FK	Recipient `users.id`
`content`	TEXT	No		Body of message
`created_at`	TIMESTAMP	No	Default Now	Transmission timestamp
`is_read`	BOOLEAN	Yes	Default FALSE	Read receipt status
`is_edited`	BOOLEAN	Yes	Default FALSE	Modification flag
`is_deleted`	BOOLEAN	Yes	Default FALSE	Soft deletion flag

13. **`materials`**

Course lecture notes, syllabi, and study resources.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK	Reference to `classes.id`
`teacher_id`	INT	No	FK	Reference to `users.id`
`title`	VARCHAR(255)	No		Material title
`description`	TEXT	Yes		Detailed overview of material
`file_path`	VARCHAR(255)	Yes		Relative storage path of uploaded asset
`created_at`	TIMESTAMP	No	Default Now	Upload timestamp

14. **`assignments`** & **`assignment_submissions`**

LMS coursework distribution and submission tracking.

- **`assignments`** **Table:**

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK	Reference to `classes.id`
`teacher_id`	INT	No	FK	Reference to `users.id`
`title`	VARCHAR(255)	No		Assignment title
`description`	TEXT	Yes		Assignment brief and requirements
`due_date`	DATETIME	No		Submission cutoff timestamp
`max_points`	DECIMAL(5,2)	Yes	Default 100.00	Maximum achievable score
`attachment_path`	VARCHAR(255)	Yes		Supplementary task sheet path
`created_at`	TIMESTAMP	No	Default Now	Publication timestamp

- **`assignment_submissions` Table:**

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`assignment_id`	INT	No	FK, Unique(1)	Reference to `assignments.id`
`student_id`	INT	No	FK, Unique(2)	Reference to `users.id`
`submission_text`	TEXT	Yes		Text response entered by student
`file_path`	VARCHAR(255)	Yes		Path of uploaded student solution file
`submitted_at`	TIMESTAMP	No	Default Now	Submission timestamp
`marks_obtained`	DECIMAL(5,2)	Yes	Default NULL	Graded score awarded
`feedback`	TEXT	Yes	Default NULL	Teacher qualitative remarks
`status`	ENUM('submitted', 'graded')	Yes	Default 'submitted'	Grading state
Constraint			**UNIQUE(assignment_id, student_id)**	One submission per student

15. `announcements`

Broadcast bulletin entity.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`class_id`	INT	No	FK	Reference to `classes.id`
`teacher_id`	INT	No	FK	Reference to `users.id`
`title`	VARCHAR(255)	No		Announcement title
`content`	TEXT	No		Announcement text body
`target_audience`	ENUM('all', 'students_only', 'parents_only')	Yes	Default 'all'	Audience filter
`created_at`	TIMESTAMP	No	Default Now	Publication timestamp

16. `student_of_the_month`

Honor roll cache entity.

Column	Type	Nullable	Key	Description
`id`	INT	No	PK, Auto	Primary key
`month_period`	VARCHAR(7)	No	Unique(1)	Evaluated calendar month (`YYYY-MM`)
`subject`	VARCHAR(100)	No	Unique(2)	Academic subject title
`student_id`	INT	No	FK	Reference to `users.id`
`average_score`	DECIMAL(5,2)	No		Highest average score achieved
`tests_count`	INT	Yes	Default 1	Total assessments evaluated
`created_at`	TIMESTAMP	No	Default Now	Calculation timestamp
Constraint			**UNIQUE KEY unique_month_subject (month_period, subject)**	Enforces one honoree per subject per month

7. Machine Learning & Algorithmic Specifications

7.1 Random Forest Dropout Predictor Architecture

- **Objective:** Detect students exhibiting early behavioral or academic disengagement indicators to permit timely pedagogical intervention.
- **Model Specification:** Scikit-Learn `RandomForestClassifier` configured with 50 decision trees (`n_estimators=50`) and a deterministic seed (`random_state=42`).
- **Feature Pipeline:**
  1. $x_1 = \text{attendance_pct}$: Total sessions attended divided by total class sessions held ($\times 100$).
  2. $x_2 = \text{recent_att_pct}$: Sessions attended in the last 30 days divided by class sessions held in the last 30 days ($\times 100$).
  3. $x_3 = \text{avg_mark_pct}$: Average percentage obtained across all assessment tests.
  4. $x_4 = \text{mark_trend}$: Mean score of the second half of recorded exams minus the mean score of the first half.
  5. $x_5 = \text{missed_hw_ratio}$: Ratio of published assignments with no submission record: $\text{missed_hw_ratio} = \frac{\text{Total Published} - \text{Total Submitted}}{\text{Total Published}}$
- **Probability Calibration & Score Formulation:** The classifier predicts probability vector $P = [p_{\text{low}}, p_{\text{medium}}, p_{\text{high}}]$. The composite risk percentage is derived using weighted probability calibration: $\text{Risk Score } (\%) = \min\left(100.0,\ (p_{\text{high}} \times 1.0 + p_{\text{medium}} \times 0.45) \times 100\right)$
- **Classification Rules:**
  - `HIGH`: $\text{Risk Score} \geq 55\% \ \vee \ x_1 < 55\% \ \vee \ x_3 < 45\%$
  - `MEDIUM`: $\text{Risk Score} \geq 25\% \ \vee \ x_1 < 75\% \ \vee \ x_3 < 60\%$
  - `LOW`: All other cases.

7.2 Ordinary Least Squares (OLS) Grade Forecaster Architecture

- **Objective:** Project a student's impending examination score and continuous grade trajectory.
- **Input Stream:** Chronologically ordered sequence of percentage scores combining regular exams and graded homework submissions: $S = [(t_0, s_0), (t_1, s_1), \ldots, (t_{k-1}, s_{k-1})], \quad \text{where } s_i \in [0.0, 100.0]$
- **Regression Formulation:** For $k \geq 2$, an OLS model fits the indices $X = [0, 1, \ldots, k-1]^T$ to observations $y = [s_0, s_1, \ldots, s_{k-1}]^T$: $m = \frac{\sum_{i=0}^{k-1}(i-\bar{x})(s_i-\bar{y})}{\sum_{i=0}^{k-1}(i-\bar{x})^2}, \quad c = \bar{y} - m\bar{x}$ The forecast for the next examination ($x = k$) is computed as: $\hat{s}_k = \max\left(0.0,\ \min\left(100.0,\ m \cdot k + c\right)\right)$
- **Trajectory Slope Diagnostics:**
  - $m > +1.5$: "Improving Trajectory"
  - $m < -1.5$: "Declining Trajectory"
  - $-1.5 \leq m \leq +1.5$: "Stable Performance"

7.3 Student of the Month Ranking Algorithm

- **Objective:** Select a premier student per subject each calendar month to foster positive academic motivation.

- **SQL Evaluation Query (MySQL 5.1 Compatible):**

```
SELECT
    c.subject,
    m.student_id,
    u.name as student_name,
    sp.grade,
    AVG(m.marks_obtained / m.max_marks * 100) as avg_score,
    COUNT(m.id) as tests_taken
FROM marks m
JOIN classes c ON m.class_id = c.id
JOIN users u ON m.student_id = u.id
LEFT JOIN student_profiles sp ON u.id = sp.user_id
WHERE DATE_FORMAT(m.date_recorded, '%Y-%m') = %s
GROUP BY c.subject, m.student_id, u.name, sp.grade
ORDER BY c.subject ASC, avg_score DESC, tests_taken DESC;
```

- **Fallback Logic:** If the active calendar month contains zero recorded marks, the query substitutes `%s` with `(SELECT DATE_FORMAT(MAX(date_recorded), '%Y-%m') FROM marks)`.
- **Deduplication Persistence:** Top candidates are written to `student_of_the_month` using `ON DUPLICATE KEY UPDATE average_score = VALUES(average_score), tests_count = VALUES(tests_count)`.

8. Verification, Validation & Acceptance Criteria

Req ID	Acceptance Test Description	Expected Result	Verification Method
FR-AUTH-01	Student registers on public `/register` form.	Profile created; 64-char QR token generated; password securely hashed.	Automated Integration Test
FR-FIN-02	Submit fee payment for Class A, Student S, Period $2026-09$ twice.	First attempt succeeds (200 OK); second attempt rejected (409 Conflict / Duplicate error).	Automated Integration Test
FR-ATT-01	Scan student QR code for Class C on Date D twice.	First attempt marks `present`; second attempt updates or notes already marked; no duplicate DB rows.	Automated Integration Test
FR-PAR-04	Parent accesses Progress Report from sidebar and executes browser print dialog.	Renders complete academic history; hides navigation chrome; displays clean printable template.	Visual / DOM Inspection
FR-SOM-01	Query Student of the Month for subject with multiple test records.	Identifies student with highest average score; breaks tie by test count; renders honor banner on dashboards.	Automated Integration Test
FR-AIR-01	Compute risk profile for student with $< 50\%$ attendance and declining marks.	Categorizes student as `HIGH` risk; lists diagnostic factors (low attendance, declining trend).	Unit / Integration Test
FR-AIG-01	Compute grade projection for student with score sequence $[60, 70, 80]$.	Regression computes positive slope ($m > 0$); projects score $\geq 85\%$ (Grade A).	Unit / Integration Test
NFR-SEC-01	Execute SQL injection attempt on login and search parameters.	Parameterized queries escape all input; zero unauthorized data exposed.	Security Audit
NFR-PERF-01	Benchmark dashboard page loads under normal conditions.	Server response delivered in under 250ms.	Performance Benchmark

End of Specification.